DATA VISUALIZATION — FINAL NOTES

Matplotlib + Seaborn | Python Data Analysis

---

1. DATA VISUALIZATION KYA HAI?

Definition: Data ko charts aur graphs ke through represent karna, taaki patterns, trends, comparisons aur outliers easily samajh aa sakein.

Example: 100 students ke marks ko individually padhne ke bajaye bar chart se compare karna.

Visualization kyun use karte hain?

- Comparison — kaun zyada ya kam hai?
- Trends — time ke saath kya change hua?
- Distribution — data kis range mein spread hai?
- Relationship — do variables ka connection hai?
- Outliers — unusual values kaun si hain?

Main libraries

Library	Use
Matplotlib	Basic plots aur detailed customization
Seaborn	Statistical plots aur attractive defaults
Pandas	Data cleaning, grouping aur analysis

NumPy	Numerical calculations aur arrays

---

2. MATPLOTLIB BASICS

2.1 Import

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

`matplotlib.pyplot` plotting functions provide karta hai.

2.2 Basic Line Plot — `plt.plot()`

Definition: Values ke beech trend ya change dikhata hai.

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [100, 150, 120, 200, 250]

plt.plot(months, sales)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)

plt.show()
```

Output: Sales Jan se May tak overall increase ho rahi hai, lekin March mein dip hai.

Syntax:

```
plt.plot(x, y)
```

- `x` — horizontal axis
- `y` — vertical axis
- `title()` — chart ka title
- `xlabel()` — X-axis label
- `ylabel()` — Y-axis label
- `grid()` — grid lines

- `show()` — chart display karta hai

2.3 Line Plot Customization

```
plt.plot(
    months,
    sales,
    color="blue",
    marker="o",
    linestyle="--",
    linewidth=2,
    label="Sales"
)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.show()
```

Important parameters:

- `color` — line ka colour
- `marker` — data points ka symbol
- `linestyle` — line ka style
- `linewidth` — line ki thickness
- `label` — legend mein naam
- `legend()` — labels display karta hai

2.4 Bar Chart — `plt.bar()`

Definition: Categories ke beech numerical values compare karta hai.

```
names = ["A", "B", "C", "D"]
marks = [80, 90, 70, 85]

plt.bar(names, marks)

plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.show()
```

Use: Department-wise sales, student marks, company profits.

Horizontal Bar Chart — `plt.barh()`

```
plt.barh(names, marks)

plt.title("Student Marks")
plt.xlabel("Marks")
plt.ylabel("Students")
plt.show()
```

Difference:

- `bar()` — vertical bars
- `barh()` — horizontal bars

2.5 Histogram — `plt.hist()`

Definition: Numerical data ka distribution dikhata hai. Data ko ranges, yaani bins, mein divide karta hai.

```
marks = [45, 50, 55, 60, 60, 65, 70, 75, 80, 85, 90]

plt.hist(marks, bins=5, edgecolor="black")

plt.title("Marks Distribution")
plt.xlabel("Marks Range")
plt.ylabel("Frequency")
plt.show()
```

Important:

- `bins=5` — data ko 5 ranges mein divide karne ki request.
- `frequency` — har range mein kitni observations hain.
- Histogram ki bars generally touch karti hain kyunki numerical ranges continuous hoti hain.

2.6 Scatter Plot — `plt.scatter()`

Definition: Do numerical variables ke beech relationship dekhne ke liye.

```
study_hours = [1, 2, 3, 4, 5, 6]
marks = [35, 40, 55, 60, 75, 90]

plt.scatter(study_hours, marks)

plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
```

```
plt.ylabel("Marks")
plt.show()
```

Interpretation: Points upward direction mein hain, toh is example mein study hours aur marks ke beech positive relationship dikh raha hai.

Yaad rakh: correlation hone se causation prove nahi hota.

2.7 Box Plot — `plt.boxplot()`

Definition: Data ka spread, median aur potential outliers samajhne mein help karta hai.

```
salary = [20000, 22000, 25000, 28000, 30000, 35000, 90000]

plt.boxplot(salary)

plt.title("Salary Distribution")
plt.ylabel("Salary")
plt.show()
```

Box plot mein:

- Middle line — median
- Box — first aur third quartile ke beech ka range (IQR)
- Whiskers — defined rule ke according non-outlier range
- Individual points — potential outliers

Outlier ka matlab automatically wrong data nahi hota.

2.8 Pie Chart — `plt.pie()`

Definition: Total mein categories ka proportional share dikhata hai.

```
labels = ["Food", "Rent", "Travel", "Other"]
expenses = [30, 40, 15, 15]

plt.pie(
    expenses,
    labels=labels,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Monthly Expenses")
plt.show()
```

- `labels` — category names
- `autopct` — percentage show karta hai

- `startangle` — chart ka starting angle

Tip: Bahut saari categories ya close values compare karne ke liye bar chart usually zyada readable hota hai.

2.9 Multiple Lines in One Chart

```
months = ["Jan", "Feb", "Mar", "Apr"]
company_a = [100, 120, 150, 170]
company_b = [90, 130, 140, 200]

plt.plot(months, company_a, marker="o", label="Company A")
plt.plot(months, company_b, marker="s", label="Company B")

plt.title("Company Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.show()
```

Use: Multiple companies, products ya categories ke trends compare karna.

2.10 Figure Size aur Subplots

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar(names, marks)
axes[0].set_title("Marks Comparison")
axes[0].set_xlabel("Students")
axes[0].set_ylabel("Marks")

axes[1].plot(months[:4], company_a, marker="o")
axes[1].set_title("Company A Sales")
axes[1].set_xlabel("Month")
axes[1].set_ylabel("Sales")

plt.tight_layout()
plt.show()
```

- `figsize=(12, 4)` — figure ki size.
- `subplots(1, 2)` — 1 row aur 2 columns mein charts.
- `axes[0]`, `axes[1]` — individual chart areas.
- `set_title()` aur `set_xlabel()` — individual Axes ko configure karte hain.
- `tight_layout()` — labels aur charts ke overlap ko reduce karta hai.

Important difference:

```
plt.title("Title")       # pyplot interface
ax.set_title("Title")    # individual Axes interface
```

2.11 Matplotlib mein Bar Labels

```
fig, ax = plt.subplots()

bars = ax.bar(names, marks)

ax.bar_label(bars)
ax.set_title("Student Marks")
ax.set_xlabel("Students")
ax.set_ylabel("Marks")

plt.show()
```

`ax` Matplotlib ka Axes object hai. `bar_label()` bar ke upar numerical values display karta hai.

---

3. SEABORN BASICS

3.1 Seaborn Kya Hai?

Definition: Seaborn, Matplotlib ke upar built statistical visualization library hai. Ye DataFrame columns ke saath convenient syntax aur attractive default styles provide karti hai.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

3.2 Sample DataFrame

Neeche ke Seaborn examples isi DataFrame ko use karenge.

```
df = pd.DataFrame({
    "name": ["Nasir", "Rahul", "Aman", "Priya", "Sara", "Karan"],
    "age": [20, 21, 19, 20, 22, 21],
    "marks": [80, 90, 70, 95, 85, 75],
    "gender": ["M", "M", "M", "F", "F", "M"],
    "dept": ["BCA", "MCA", "BCA", "MCA", "BCA", "BCA"],
    "hours": [3, 5, 2, 6, 4, 3],
    "salary": [25000, 40000, 22000, 45000, 35000, 28000]
})
```

```
print(df)
```

DataFrame ke columns:

- `name`, `gender`, `dept` — categorical data
- `age`, `marks`, `hours`, `salary` — numerical data

3.3 Seaborn Theme — `sns.set_theme()`

```
sns.set_theme(style="whitegrid", palette="deep")
```

- `style="whitegrid"` — white background with grid lines.
- `palette="deep"` — default categorical colour palette.

Ye optional hai. Iske bina bhi charts banenge.

Common styles: `"whitegrid"`, `"darkgrid"`, `"white"`, `"dark"`, `"ticks"`.

Common palettes: `"deep"`, `"bright"`, `"muted"`, `"pastel"`, `"Set2"`.

3.4 Seaborn Line Plot — `sns.lineplot()`

Definition: Trends aur numerical values mein changes dikhata hai.

```
sales_df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar", "Apr"],
    "sales": [100, 150, 130, 200]
})

sns.lineplot(data=sales_df, x="month", y="sales", marker="o")

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

- `data` — source DataFrame
- `x` — X-axis column
- `y` — Y-axis column
- `marker` — points ka symbol

Seaborn mein `data=df` dene par column names directly pass kar sakte hain.

```
sns.lineplot(data=df, x="hours", y="marks")
plt.show()
```

Lineplot mein observations ko connect karne se pehle check karo ki X-axis ka meaningful order hai. Unordered categories ke liye bar ya scatter plot zyada suitable ho sakta hai.

3.5 `hue` — Categories ko Colours se Separate Karna

Definition: `hue` kisi categorical column ke according observations ko different colours/groups mein divide karta hai.

```
sns.scatterplot(
    data=df,
    x="hours",
    y="marks",
    hue="gender"
)

plt.title("Study Hours vs Marks by Gender")
plt.show()
```

Yahan:

- X-axis — study hours
- Y-axis — marks
- `hue="gender"` — male/female observations ko different colours mein dikhata hai.
- Legend categories ko identify karne mein help karti hai.

`hue` ko lineplot, scatterplot, barplot aur countplot jaise functions mein use kar sakte hain.

Ek aur example

```
sns.barplot(
    data=df,
    x="dept",
    y="salary",
    hue="gender"
)

plt.title("Department-wise Salary by Gender")
plt.show()
```

`hue` visualization ko groups mein divide karta hai; ye apne aap total ya average calculate karne ka naam nahi hai.

3.6 Count Plot — `sns.countplot()`

Definition: Har category mein kitni rows/observations hain, ye count karta hai.

```
sns.countplot(data=df, x="dept")

plt.title("Students per Department")
plt.xlabel("Department")
```

```
plt.ylabel("Number of Students")
plt.show()
```

DataFrame mein BCA ki 4 aur MCA ki 2 rows hain. Isliye BCA ka bar 4 aur MCA ka bar 2 dikhayega.

Countplot with Hue

```
ax = sns.countplot(data=df, x="dept", hue="gender")

for container in ax.containers:
    ax.bar_label(container)

plt.title("Department-wise Gender Count")
plt.show()
```

- `x="dept"` — department categories.
- `hue="gender"` — har department ke andar gender-wise counts.
- `ax.containers` — bar groups.
- `ax.bar_label(container)` — bars par counts show karta hai.

Note: Countplot rows count karta hai; ye marks ya salary ka average nahi nikalta.

3.7 Histogram — `sns.histplot()`

Definition: Numerical data ka distribution bins mein dikhata hai.

```
sns.histplot(data=df, x="marks", bins=5, kde=True)

plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

- `bins=5` — 5 bins request karta hai.
- `kde=True` — distribution ki smoothed density curve add karta hai.

Difference:

- `plt.hist()` — Matplotlib histogram.
- `sns.histplot()` — Seaborn histogram, jisme DataFrame integration aur KDE jaise options milte hain.

3.8 Bar Plot — `sns.barplot()`

Definition: Har category ke numerical variable ka default mein mean (average) compare karta hai.

```
sns.barplot(data=df, x="dept", y="marks")

plt.title("Average Marks by Department")
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.show()
```

Important: `barplot()` aur `countplot()` same nahi hain.

- `countplot()` — har category mein kitni rows hain.
- `barplot()` — har category mein numerical variable ka default mean.

Gender-wise comparison

```
sns.barplot(
    data=df,
    x="dept",
    y="marks",
    hue="gender",
    errorbar=None
)

plt.title("Average Marks by Department and Gender")
plt.show()
```

`errorbar=None` error bars hide karta hai. Ye average ko change nahi karta.

Sum calculate karna

Agar category-wise sum dikhana hai:

```
sns.barplot(
    data=df,
    x="dept",
    y="salary",
    estimator="sum",
    errorbar=None
)

plt.title("Total Salary by Department")
plt.show()
```

`estimator="sum"` available observations ka sum calculate karta hai, jabki default estimator mean hai.

3.9 Box Plot — `sns.boxplot()`

```
sns.boxplot(data=df, x="dept", y="salary")

plt.title("Salary Distribution by Department")
plt.show()
```

Use: Departments ya groups ki salary distribution, median aur potential outliers compare karna.

3.10 Scatter Plot — `sns.scatterplot()`

```
sns.scatterplot(data=df, x="hours", y="marks")

plt.title("Study Hours vs Marks")
plt.show()
```

Use: Do numerical variables ka relationship dekhna.

Extra parameter:

```
sns.scatterplot(
    data=df,
    x="hours",
    y="marks",
    hue="dept",
    size="salary"
)

plt.show()
```

Yahan `hue` categories ke colours aur `size` marker size ko salary ke according vary karne deta hai.

3.11 Heatmap — `sns.heatmap()`

Definition: Values ko colours ke through matrix format mein represent karta hai. Correlation matrix visualize karne mein commonly used hai.

```
corr = df[["age", "marks", "hours", "salary"]].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)
```

```
plt.title("Correlation Heatmap")
plt.show()
```

- `.corr()` — numerical columns ke pairwise correlations calculate karta hai.
- `annot=True` — cells mein values show karta hai.
- `cmap` — colour mapping.
- `fmt=".2f"` — values ko 2 decimal places mein display karta hai.
- `vmin=-1`, `vmax=1` — correlation scale ki limits.

Correlation interpretation:

- `+1` — perfect positive linear relationship.
- `0` — no linear correlation.
- `-1` — perfect negative linear relationship.

Correlation causation prove nahi karta.

3.12 KDE Plot — `sns.kdeplot()`

```
sns.kdeplot(data=df, x="marks", fill=True)

plt.title("Marks Density")
plt.show()
```

KDE ek smooth estimate deta hai ki numerical values kis range mein concentrated hain.

- Histogram actual bin counts dikhata hai.
- KDE estimated probability density curve dikhata hai.

3.13 Pairplot — `sns.pairplot()`

```
sns.pairplot(
    df[["marks", "hours", "salary", "gender"]],
    hue="gender"
)

plt.show()
```

Multiple numerical variables ke pairwise relationships aur individual distributions ek saath explore karne ke liye useful hai.

Tip: Bade datasets mein pairplot slow aur cluttered ho sakta hai; pehle relevant columns select karo.

3.14 Seaborn Plot Par Labels Lagana

```
ax = sns.lineplot(
    data=sales_df,
    x="month",
    y="sales",
    marker="o"
)

for x, y in zip(sales_df["month"], sales_df["sales"]):
    ax.annotate(
        str(y),
        xy=(x, y),
        xytext=(0, 8),
        textcoords="offset points",
        ha="center"
    )

plt.show()
```

- `ax` — Seaborn se returned Matplotlib Axes object.
- `annotate()` — specific point ke paas text add karta hai.
- `xy` — point ki location.
- `xytext` — text ka offset.
- `ha="center"` — text horizontally center align hota hai.

Bars ke liye `bar_label()`; line/scatter ke selected points ke liye `annotate()` ya `text()` useful hai.

---

4. GROUPBY VS HUE

Ye Data Analysis mein bahut important difference hai.

`groupby()` — Calculation

```
department_salary = df.groupby("dept")["salary"].sum()

print(department_salary)
```

Ye har department ka total salary calculate karta hai.

Multiple categories ke saath:

```
result = df.groupby(["dept", "gender"])["salary"].mean()

print(result)
```

Ye department aur gender ke combinations ka average salary calculate karta hai.

`hue` — Visualization

```
sns.barplot(
    data=df,
    x="dept",
    y="salary",
    hue="gender",
    errorbar=None
)

plt.show()
```

Ye department ke andar gender groups ko alag colours mein display karta hai aur default mein har group ka average salary dikhata hai.

Yaad rakh:

- `groupby()` — data ko group karke calculation karta hai.
- `hue` — chart mein categories ko visually separate karta hai.
- Dono ek hi analysis mein use ho sakte hain.

Example workflow:

```
summary = df.groupby("dept", as_index=False)["salary"].sum()

sns.barplot(data=summary, x="dept", y="salary", errorbar=None)

plt.show()
```

Pehle Pandas se summary calculate hui; phir Seaborn se visualize ki gayi.

---

5. KAUN SA CHART KAB USE KARNA HAI?

Business question	**Recommended chart**
Categories compare karni hain	Bar plot

Time ke saath trend dekhna hai	Line plot
Numerical distribution samajhna hai	Histogram
Do numerical variables ka relation	Scatter plot
Spread aur outliers dekhne hain	Box plot
Total mein percentage share	Pie chart, limited categories
Numerical correlation compare karna hai	Heatmap
Smooth distribution estimate karna hai	KDE plot
Multiple numerical	Pairplot

relationships explore karne hain	
Har category mein records count karne hain	Countplot

Chart choose karne se pehle apne aap se pooch:

1. Kya compare karna hai?
2. Kya trend dekhna hai?
3. Kya distribution samajhna hai?
4. Kya relationship ya correlation dekhna hai?
5. Kya data aggregated hai ya raw?

Chart business question ke according select karna hai, sirf attractive dikhne ke liye nahi.

---

6. COMPLETE MINI PROJECT — COMPANY SALES ANALYSIS

Is project mein Pandas, Matplotlib aur Seaborn combine karenge.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Create dataset
data = {
    "year": [2022, 2023, 2024, 2022, 2023, 2024],
    "company": ["A", "A", "A", "B", "B", "B"],
    "revenue": [100, 120, 150, 90, 130, 160],
    "profit": [10, 15, 20, 12, 18, 25]
}

df = pd.DataFrame(data)

sns.set_theme(style="whitegrid", palette="deep")
```

```
    # 2. Inspect data
```

Analysis kaise karna hai?

- Revenue trend se sales ki direction compare karo.
- Profit chart se actual profit compare karo.
- Profit margin se revenue ke proportion mein profitability compare karo.
- Final recommendation se pehle debt, cash flow, valuation aur data quality bhi investigate karo.

Important: Ye sirf practice ke liye fictional data hai. Isse kisi real company mein investment ka decision nahi lena chahiye.

---

7. IMPORTANT DIFFERENCES — QUICK REVISION

Concept A	Concept B	Difference
`plt.plot()`	`sns.lineplot()`	Matplotlib basic line plot; Seaborn statistical defaults aur DataFrame integration
`plt.bar()`	`sns.barplot()`	Basic bars vs category-wise numerical aggregation, default mean

`plt.hist()`	`sns.histplot()`	Dono histograms; Seaborn mein KDE aur convenient hue options
`countplot()`	`barplot()`	Row counts vs numerical statistic
`groupby()`	`hue`	Calculation vs visual grouping
`plt.title()`	`ax.set_title()`	Pyplot interface vs Axes-specific interface
`bar_label()`	`annotate()`	Bars par labels vs selected positions par text
Histogram	Bar chart	Numerical ranges vs categorical comparison

---

8. COMMON MISTAKES

1. `plt.show()` bhool jaana

Script mein chart display karne ke liye generally `plt.show()` use karo. Jupyter kabhi-kabhi charts automatically display karta hai.

2. Wrong chart select karna

Time-series trend ke liye line plot aur category comparison ke liye bar chart usually better starting choices hain.

3. `hue` ko aggregation samajhna

`hue` groups ko colours se separate karta hai; `barplot()` apne estimator ke according statistic calculate karta hai.

4. Histogram aur bar chart ko same samajhna

Histogram numerical intervals ki frequency dikhata hai; bar chart categories compare karta hai.

5. Correlation ko causation samajhna

Do variables related hon, iska matlab ye nahi ki ek doosre ko cause karta hai.

6. Har chart ko customize karna zaroori samajhna

Pehle chart ka purpose clear rakho. Colours aur styling baad mein.

7. Missing values ignore karna

Analysis se pehle `.isnull().sum()` se missing values inspect karo.

```
print(df.isnull().sum())
print(df.duplicated().sum())
```

---

9. FINAL CHEAT SHEET

```
# Matplotlib
plt.plot(x, y)
plt.bar(x, y)
plt.barh(x, y)
plt.hist(x, bins=10)
plt.scatter(x, y)
plt.boxplot(x)
plt.pie(values, labels=labels, autopct="%1.1f%%")

# Seaborn
sns.set_theme(style="whitegrid", palette="deep")
sns.lineplot(data=df, x="x", y="y", hue="category")
```

```
sns.barplot(data=df, x="category", y="value")
sns.countplot(data=df, x="category", hue="group")
sns.histplot(data=df, x="value", bins=10, kde=True)
sns.scatterplot(data=df, x="x", y="y", hue="category")
sns.boxplot(data=df, x="category", y="value")
```

FINAL LEARNING ORDER

1. Matplotlib: line, bar, histogram, scatter, boxplot, pie.
2. Titles, labels, legend, grid, colours aur markers.
3. Seaborn: lineplot, barplot, countplot, histplot, scatterplot, boxplot.
4. `hue`, `palette`, `set_theme`, `bar_label`.
5. `groupby()` ke saath visualization.
6. Heatmap, KDE aur pairplot.
7. Ek complete dataset lekar business questions answer karna.

Data Analyst rule: Pehle question samajh, phir data inspect kar, phir chart choose kar aur end mein insight explain kar. Sirf graph bana dena analysis complete karna nahi hai.